

Project RENDERING — Code Explainer

What to Know If Asked About the Implementation

Hossein F. · Christian C. · Abraham V.

2026

How the App is Structured

Three distinct layers work together in sequence:

React UI State (`App.tsx`) → **Three.js Renderer** (`Scene.tsx`) → **Python Logic**
(`rendering_logic.py` via Pyodide)

State flows one direction — settings change in the UI, get passed as props to `Scene`, which re-renders the Three.js objects. There is no two-way binding and no shared mutable state between layers.

How State is Managed

`SceneSettings` is one flat object in `App.tsx`. Every control (slider, toggle, color picker) calls `updateSettings()`, which merges only the changed key using a **functional update**:

```
setSettingsState(s => ({ ...s, ...newSettings }));
```

This is critical — it reads the *current* state at the time React processes the update, not the state captured in the closure when the function was defined. Without this, async operations (like the Python shading call) could complete and overwrite unrelated settings that changed in the meantime.

Why Python Runs in the Browser

Pyodide compiles CPython to WebAssembly. There is no server and no backend — the Python interpreter lives entirely inside the browser tab. `initPython()` loads it once asynchronously on startup; subsequent calls (`runPythonShading` , `parseOBJInPython`) reuse the same instance. The status indicator in the header (“Python Ready” / “Loading Python...”) reflects this initialization lifecycle.

What the Python Actually Does

Two things:

1. OBJ Parsing — reads vertex (`v`) and face (`f`) lines from a `.obj` file string, strips texture/normal indices (e.g. `1/2/3` → `1`), and fan-triangulates n-gon faces into triangles:

```
# Fan triangulation: [0,1,2,3,4] → [0,1,2], [0,2,3], [0,3,4]
for i in range(1, len(indices) - 1):
    self.faces.append([indices[0], indices[i], indices[i + 1]])
```

2. `calculate_shading` — multiplies each RGB channel by the light intensity. It is a simplified demonstration of the $I_l \cdot k_d$ term, not what actually shades the 3D scene. The 3D scene uses `objectColor` directly; `shadedColor` from Python is stored in state but only displayed in the stats panel.

How OBJ Geometry Gets Into Three.js

Python returns raw `vertices` (array of `[x, y, z]`) and `faces` (array of triangle index triples). Three.js then:

```
const positions = new Float32Array(objData.vertices.flat());
geom.setAttribute('position', new THREE.BufferAttribute(positions, 3));
geom.setIndex(new THREE.BufferAttribute(indices, 1));
geom.computeVertexNormals();
```

1. Flattens vertices into a `Float32Array` → `BufferAttribute`
2. Sets face indices via `geom.setIndex()`
3. Calls `geom.computeVertexNormals()` — this averages face normals at shared vertices, which is what makes shading look smooth across the mesh
4. Centers and scales to fit the scene (bounding box normalization) so any OBJ model appears at a consistent size

Why Wireframe Forces an Unlit Material

If you set `wireframe: true` on `meshPhongMaterial`, Three.js still runs the full lighting shader — but now on edge geometry instead of face geometry. Edge triangles have completely different normals, so $N \cdot L$ produces wildly different values and the scene appears to change its entire lighting when wireframe is toggled.

The fix — detect wireframe first and return `meshBasicMaterial` before checking the shading model:

```
function ShadedMaterial({ shadingModel, color, wireframe, roughness,
  metalness }) {
  if (wireframe) return <meshBasicMaterial color={color} wireframe />;
  // ... rest of shading model switch
}
```

`meshBasicMaterial` is unlit — it ignores all light sources by definition, so the wireframe appearance is always consistent.

How the Shading Model Controls the Three.js Material

`ShadedMaterial` in `Scene.tsx` is a straightforward switch between Four.js material classes:

UI Selection	Three.js Material	Notes
Unlit	<code>meshBasicMaterial</code>	No lighting computed
Lambert	<code>meshLambertMaterial</code>	Per-vertex diffuse (Gouraud)
Blinn-Phong	<code>meshPhongMaterial</code>	Per-fragment diffuse + specular
PBR / GGX	<code>meshStandardMaterial</code>	Microfacet BRDF

For Blinn-Phong, roughness maps to shininess: $n = \max(1, \text{round}((1 - \text{roughness}) \times 300))$. For PBR, `roughness` and `metalness` are passed directly to `meshStandardMaterial`.

Why Some Sliders are Greyed Out

`Controls.tsx` computes three boolean flags from the current shading model:

```
const isLit      = settings.shadingModel !== 'basic';
const hasSpecular = settings.shadingModel === 'phong' ||
  settings.shadingModel === 'standard';
const hasPBR     = settings.shadingModel === 'standard';
```

Sliders receive `opacity-40 pointer-events-none` via Tailwind when the active model does not use that parameter. For example:

- **Lambert** — roughness and metalness are greyed out (no specular term to plug them into)
- **Blinn-Phong** — metalness is greyed out (no Fresnel equation, no conductor model)
- **Unlit** — all lighting controls are greyed out (material ignores all light sources)

How Screenshots Work

The WebGL canvas is created with `preserveDrawingBuffer: true` :

```
<Canvas gl={{ preserveDrawingBuffer: true }}>
```

Without this flag, WebGL clears the framebuffer immediately after each frame is composited to the screen, making `toDataURL()` return a blank image. A ref exposes the `WebGLRenderer` to `App.tsx` . On click:

```
const url = gl.domElement.toDataURL('image/png');
const a = document.createElement('a');
a.href = url;
a.download = 'render.png';
a.click();
```

This programmatically triggers a browser download without navigating away from the page.

How the Camera Works

`OrbitControls` from `@react-three/drei` wraps Three.js's orbit controller. It stores spherical coordinates (radius, θ , φ) around a target point and translates mouse/touch input into camera movement:

Input	Action
Left drag	Rotate — changes θ and φ around the target
Right drag	Pan — translates both camera and target
Scroll	Zoom — changes the radial distance
R key	Reset — calls <code>orbitRef.current?.reset()</code>

Input	Action
Space	Toggle auto-rotate
W	Toggle wireframe

`enableDamping` with `dampingFactor: 0.05` adds inertia — each frame the angular velocity decays by 5%, so the camera coasts to a stop rather than halting instantly.

`makeDefault` on both `PerspectiveCamera` and `OrbitControls` registers them as the active camera and controller for the entire React Three Fiber scene tree.